

Datamanagement & Analytics

Prof. Dr. André Drews

Fachbereich Maschinenbau und Wirtschaft

18. Mai 2026

- nur für den Gebrauch innerhalb des Kurses -

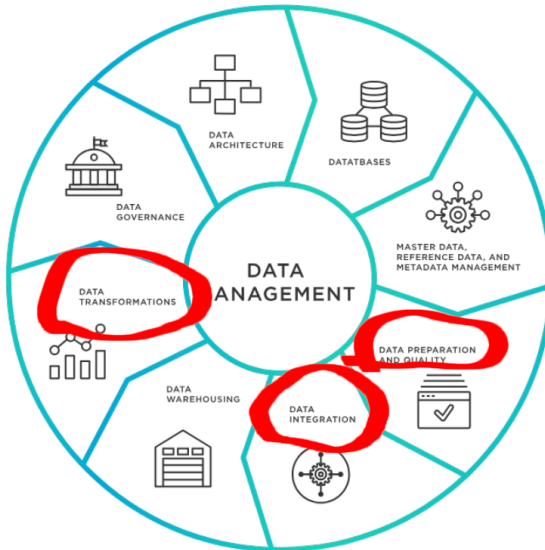
Vorlesungskapitel 6

- Data Integration, Data Transformation , Data Warehouse, Data Lake -

Lernziele

- Verständnis über Data Integration und Data Transformation
- Anwendungskompetenz im Bereich Data Warehousing und Data Lake

1. Data Integration und Data Transformation
2. Data Warehouse
3. Data Integration, Extract Transform Load (ETL)
4. Data Lake



Wissenschaftliche Definition

Data Integration bezeichnet die technische und organisatorische Zusammenführung von Daten aus unterschiedlichen, heterogenen Quellen und Systemen, um eine konsistente, gemeinsame und nutzbare Datenbasis bereitzustellen.

Ziel:

- Daten gemeinsam nutzbar machen
- Datensilos vermeiden
- Konsistente Informationsbasis schaffen

Supply-Chain-Bezug

In digitalen Supply Chains umfasst Data Integration die Verbindung von ERP-, Lager-, Transport-, IoT- und Cloud-Systemen, um Echtzeit-Transparenz und datengetriebene Entscheidungen zu ermöglichen.

Typische integrierte Systeme:

- ERP-Systeme (z.B. SAP)
- Warehouse Management Systems (WMS)
- Transport Management Systems (TMS)
- IoT-Sensoren
- Cloud-Plattformen

Data Integration umfasst typischerweise:

- Datenextraktion
- Datentransformation
- Datenvereinheitlichung
- Datenübertragung
- Synchronisation

Ziel der Datenintegration:

Gemeinsame Nutzung der Daten

Typische Beispiele für Datenextraktion

Datenquelle	Extraktionsmethode	Beispiel
SQL-Datenbank	SQL Query	Kundendaten aus ERP laden
CSV-Datei	Dateiimport	Bestelldaten aus CSV lesen
REST API	HTTP Request	Wetterdaten abrufen
Sensor / IoT	Streaming	Temperaturdaten empfangen
Cloud Storage	Cloud API	Dateien aus AWS S3 laden
Logs	Log Parsing	Serverlogs analysieren
Excel-Dateien	Excel Import	Finanzdaten einlesen

Python-Beispiele für SQL-Datenbank und csv-Datei Extraktion

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("erp.db")

query = """
SELECT *
FROM customers
"""

customers = pd.read_sql(query, conn)

print(customers.head())
```

```
import pandas as pd

orders = pd.read_csv("orders.csv")

print(orders.head())
```

- nur für den Gebrauch innerhalb der Vorlesung -

Python-Beispiele für REST API und REST API Extraktion

```
import requests

response = requests.get(
    "https://api.weather.com/data"
)

data = response.json()

print(data)
```

```
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    'sensor_data',
    bootstrap_servers='localhost:9092'
)

for message in consumer:
    print(message.value)
```

Python-Beispiele für Sensor-Daten und Cloud Storage Extraktion

```
import boto3

s3 = boto3.client('s3')

s3.download_file(
    'data-bucket',
    'orders.csv',
    'orders_local.csv'
)
```

Typische Beispiele für Datentransformation

Transformation	Beschreibung	Beispiel
Formatumwandlung	Datenformate vereinheitlichen	Datum in ISO-Format umwandeln
Datenbereinigung	Fehlerhafte Werte korrigieren	Fehlende Werte entfernen
Aggregation	Daten zusammenfassen	Tagesumsätze berechnen
Filterung	Relevante Daten auswählen	Nur aktive Kunden laden
Normalisierung	Einheitliche Skalierung	Währungen vereinheitlichen
Zusammenführung	Datenquellen kombinieren	Kunden- und Bestelldaten verknüpfen
Anreicherung	Zusätzliche Informationen ergänzen	Wetterdaten hinzufügen
Kodierung	Kategorien numerisch darstellen	Länder-Codes erzeugen
Deduplizierung	Dubletten entfernen	Doppelte Kundeneinträge löschen

Typische Beispiele für Datenvereinheitlichung

Vereinheitlichung	Beschreibung		Beispiel
Datumsformat	Einheitliches Datumsformat		01.01.25 → 2025-01-01
Währungen	Gemeinsame Währung nutzen		USD → EUR
Einheiten	Maße vereinheitlichen		kg und lb zusammenführen
Bezeichner	Einheitliche IDs verwenden		Cust_ID = Customer_ID
Schreibweisen	Schreibweisen harmonisieren		München = Muenchen
Ländercodes	Standardisierte Codes nutzen		DE statt Germany
Kategorien	Einheitliche Kategorien definieren		M / Medium zusammenführen
Zeitzone	Zeitangaben vereinheitlichen		UTC verwenden
Datenmodelle	Gemeinsames Schema erzeugen		Unterschiedliche Tabellenstrukturen angleichen

Typische Beispiele für Datenübertragung

Übertragungsart	Beschreibung		Beispiel
Dateiübertragung	Austausch über Dateien		CSV-Datei per FTP übertragen
API-Kommunikation	Direkte Systemkommunikation		Daten über REST API senden
Streaming	Kontinuierlicher Datenstrom		Sensordaten via Kafka übertragen
Datenbank-Replikation	Synchronisierung von Datenbanken		Datenbank-Spiegelung
Cloud Transfer	Übertragung in Cloud-Systeme		Upload nach AWS S3
Message Broker	Nachrichtenbasierte Kommunikation		RabbitMQ / Kafka verwenden
Webhooks	Ereignisgesteuerte Übertragung		Bestellung automatisch senden
E-Mail Transfer	Versand strukturierter Daten		XML-Anhang per E-Mail
EDI	Standardisierter Datenaustausch		Elektronische Rechnungen übertragen

Typische Beispiele für Synchronisation

Synchronisation	Beschreibung	Beispiel
Datenbank-Synchronisation	Datenbanken aktuell halten	Replikation zwischen Servern
Dateisynchronisation	Dateien automatisch abgleichen	Cloud-Ordner synchronisieren
Echtzeit-Synchronisation	Sofortige Aktualisierung	Live-Tracking aktualisieren
Batch-Synchronisation	Periodischer Datenabgleich	Nächtlicher Datenabgleich
Bidirektionale Synchronisation	Änderungen in beide Richtungen	CRM und ERP synchronisieren
Cloud-Synchronisation	Lokale und Cloud-Daten angleichen	OneDrive / AWS Sync
Kalender-Synchronisation	Termine automatisch abgleichen	Google Calendar Sync
Geräte-Synchronisation	Daten zwischen Geräten teilen	Smartphone und Laptop synchronisieren

Typische Beispiele für gemeinsame Datennutzung

Art der Nutzung	Beschreibung	Beispiel
Gemeinsame Datenbank	Mehrere Systeme greifen auf dieselben Daten zu	Zentrales ERP-System
Data Warehouse	Einheitliche Analyseplattform	BI-Reports aus zentralem Warehouse
Cloud-Plattform	Daten organisationsweit verfügbar machen	Gemeinsame Nutzung über AWS/Azure
Dashboards	Gemeinsame Visualisierung von Daten	Power BI Dashboard
API-basierte Nutzung	Systeme nutzen dieselben Datenservices	CRM greift auf Kundendaten zu
Data Lake	Gemeinsamer Zugriff auf Rohdaten	Speicherung großer Datenmengen
Kollaborative Plattformen	Gemeinsame Bearbeitung von Daten	SharePoint / Google Workspace
Machine Learning	Gemeinsame Nutzung für Modelle	Trainingsdaten für KI-Systeme

Die Wahl der Integrations-Technologie hängt von den technischen Anforderungen ab.

Anforderung	Bedeutung
Datenmenge	Wie viele Daten verarbeitet werden müssen
Datenrate	Wie schnell Daten erzeugt werden
Echtzeitanforderung	Sofortige oder periodische Verarbeitung
Skalierbarkeit	Wachstum von Daten und Systemen
Systemkomplexität	Anzahl beteiligter Systeme
Verarbeitungsmodell	Batch oder kontinuierliche Verarbeitung

Anforderung	Technologie
Große Datenmengen	ETL, Spark
Sehr hohe Datenraten	Kafka / Streaming
Echtzeitverarbeitung	Kafka, Event Streaming
Periodische Verarbeitung	Batch ETL
Direkte Kommunikation	APIs
Verteilte Systeme	Cloud Integration
Historische Analysen	Data Warehouse + ETL
Kontinuierliche Datenströme	Streaming Technologien

Kernaussage

Die Integrations-Technologie wird insbesondere durch Datenmenge, Geschwindigkeit, Echtzeit- und Skalierungsanforderungen bestimmt.

Python-Beispiel für ETL - Extract

```
import pandas as pd
import sqlite3

# =====
# EXTRACT
# =====

# CSV-Datei laden
customers = pd.read_csv("customers.csv")

print("Daten geladen:")
print(customers.head())
```

Python-Beispiel für ETL - Transform

```
# =====  
# TRANSFORM  
# =====  
  
# Fehlende Werte entfernen  
customers = customers.dropna()  
  
# Namen vereinheitlichen  
customers["country"] = (  
    customers["country"]  
    .str.upper()  
)  
  
# Datumsformat umwandeln  
customers["signup_date"] = pd.to_datetime(  
    customers["signup_date"]  
)  
  
print("Daten transformiert")
```

Python-Beispiel für ETL - Load

```
# =====  
# LOAD  
# =====  
  
# Verbindung zur Datenbank  
conn = sqlite3.connect("warehouse.db")  
  
# Daten ins Warehouse laden  
customers.to_sql(  
    "customers_clean",  
    conn,  
    if_exists="replace",  
    index=False  
)  
  
print("Daten gespeichert")
```

Python-Beispiel für Kafka Event-Streaming

```
from kafka import KafkaConsumer
import json

# Verbindung zu Kafka
consumer = KafkaConsumer(
    'temperature_sensor',
    bootstrap_servers='localhost:9092',
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))
)

print("Warte auf Temperaturdaten...")
```

Python-Beispiel für Kafka Event-Streaming

```
# Nachrichten kontinuierlich empfangen
for message in consumer:

    data = message.value

    sensor_id = data["sensor_id"]
    temperature = data["temperature"]

    print(
        f"Sensor: {sensor_id}, "
        f"Temperatur: {temperature}°C"
    )

# Alarmbedingung
if temperature > 8:

    print("ALARM: Temperatur zu hoch!")
```

Prozess	Integrationsart
Monatsreport	Batch
GPS Tracking	Echtzeit
Forecasting	Batch
Expected Time Of Arrival (ETA) Prediction	Echtzeit
Stammdaten	Batch

- Batch bedeutet technisch:
Daten werden gesammelt und zu bestimmten Zeitpunkten gemeinsam verarbeitet.
- Echtzeit bedeutet technisch:
Daten werden unmittelbar nach ihrem Eintreffen verarbeitet.

SAP → Batch ETL → Warehouse

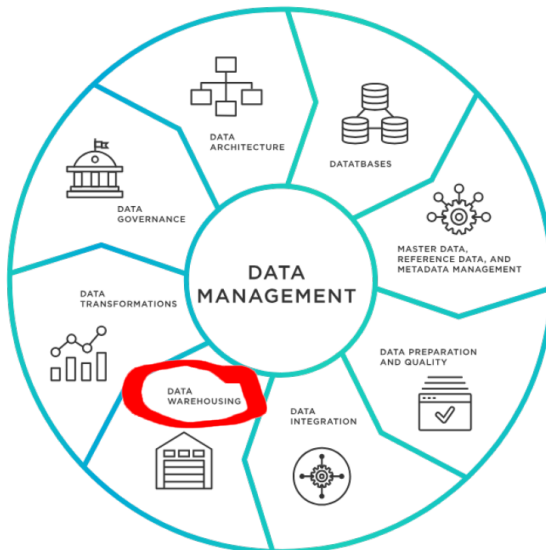
GPS → Kafka → Live Dashboard

IoT → Event Stream → Alarm

Alle Systeme → Analytics Plattform

1. Data Integration und Data Transformation
4. Grundlagen Datawarehouse
3. Data Lake

Workflow Datenaquise - Datawarehouse



Datawarehouse

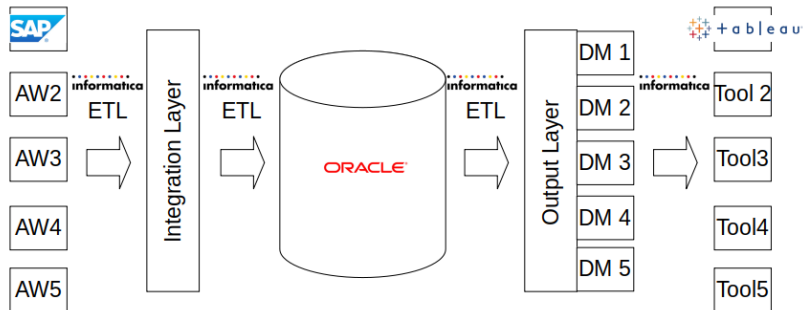
- Ein Data Warehouse (Daten Warenlager) ist eine für Analysezwecke optimierte Zentrale, die Daten aus mehreren, meistens heterogenen Quellen (Firmenanwendungen) zusammenführt. Ohne ein Datawarehouse wäre eine Gesamtanalyse nicht möglich, da die Daten unternehmensweit verteilt wären.
- Die externen Daten werden über einen Prozess (Extract Transform Load (ETL)) auf ein einheitliches Format gebracht und in das Datawarehouse importiert. Meistens handelt es sich bei dem Datawarehouse um eine oder viele relationale Datenbank(en).

Historie:

- Der Begriff "Data Warehouse" wurde erstmals 1988 von Barry Devlin verwendet.

- Ein Datawarehouse wird typischerweise täglich geupdated → die Analyse hat mindestens einen Tag Zeitversatz.
- Alle operational Prozesse des Datawarehouse sind automatisiert (z.B. batch-Programme, die jede Nacht um 23:59 Uhr Daten aus den Datenbanken der Anwendungsprogramme importieren).

Architektur Datawarehouse

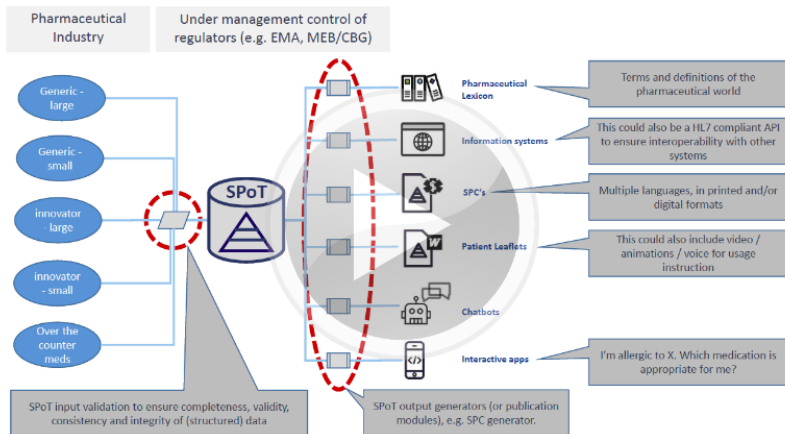


- **Controlling:**
 - ▶ Erstellung von reports aus dem Datawarehouse.
- **Business Intelligence:**
 - ▶ Arbeit/Analysen/Reports von Business Intelligence/Business Analysis.
- **Data Science:**
 - ▶ Datenbezugsquelle für Klassifikation, Predictions, aufwändige reportings ...
 - ▶ Das DWH wird neben anderen Quellen (Datalake, ...) genutzt.

- Kosten- und Ressourcenermittlung
- Analyse von Geschäfts- und Produktionsprozessen
- Bereitstellung von Reports und Statistiken
- Ermittlung von Unternehmenskennzahlen
- Bereitstellung von Daten für weitergehende Analysen und Data Mining
- Strukturierung und Harmonisierung von Datenbeständen für eine globale Unternehmenssicht

Definition SPOT

- Ein Punkt in einem Datenspeicher, der die Daten korrekt und widerspruchsfrei enthält (den "wahren" Zustand der Daten). An diesem Punkt geben die Daten die Realität, die sie modellieren also korrekt wieder.
- SPOT ist vor allem dann wichtig, wenn Daten redundant gehalten werden.
- Data Warehouses enthalten kopierte und redundante Daten. Konsistente Dateninhalte aus beliebigen Quellen und durch ETL bereinigt, können ganz oder teilweise als SPOT definiert werden.
- Wenn alle Applikationen auf diesen integrierten Datenbestand zugreifen, können Inkonsistenzen vermieden werden.



Quelle: <https://www.dragon1.com/watch/470015/single-point-of-truth-spot> , Februar 2020

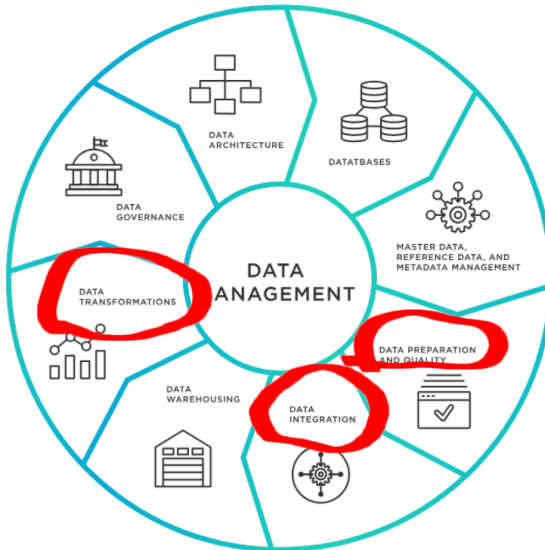
Warum Relationale Datenbank für Datawarehouse?

- Die starken Restriktionen des RDBMS (foreign keys, ...) und Konzepte wie ACID
 - ▶ Atomisticity: Eine Transaktion wird entweder ganz oder gar nicht abgeschlossen.
 - ▶ Consistency: Eine Transaktion hinterlässt einen konsistenten Datenbankzustand.
 - ▶ Isolation: Nebenläufige Transaktionen können sich nicht gegenseitig beeinflussen.
 - ▶ Durability: Daten werden nach erfolgreicher Transaktion dauerhaft in Datenbank gespeichert.

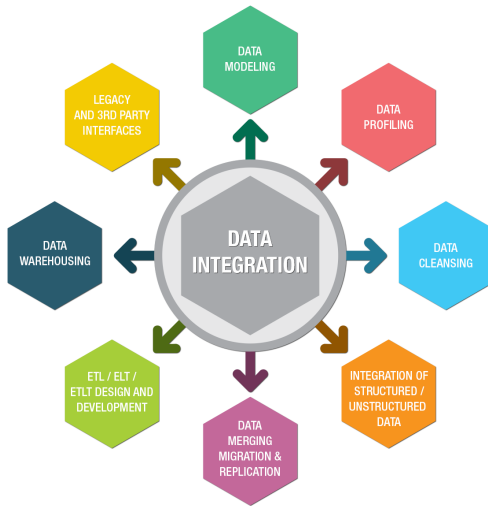
ermöglichen eine kontrollierte und sichere Datenhaltung.

- Inkonsistenzen können durch RDBMS schon beim Einfügen der Daten aufgrund von Integritätsverletzungen erkannt werden.
- Durch das RDBMS kann am ehesten ein SPOT erreicht werden.

1. Wiederholung des letzten Veranstaltungskapitels
2. Datawarehouseing
3. Data Integration / Extract Transform Load (ETL)
4. Data Lake



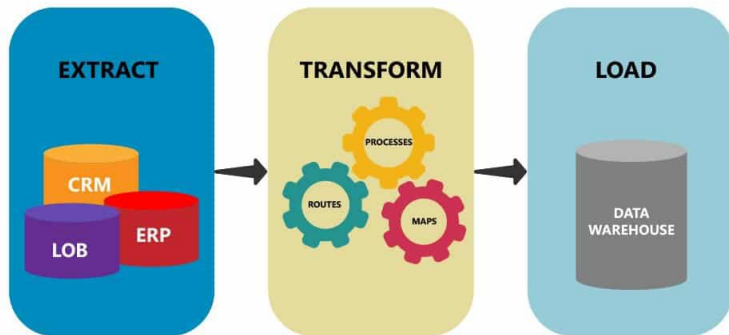
Data Integration



Definition ETL

Extract, Transform, Load (ETL) ist ein Prozess, bei dem Daten aus mehreren gegebenenfalls unterschiedlich strukturierten Datenquellen in einer Zieldatenbank vereinigt werden.

- **Extraktion** der relevanten Daten aus heterogenen Quellen
- **Transformation** der Daten in das Schema und Format der Zieldatenbank
- **Laden** der Daten in die Zieldatenbank



ETL - Extract, Transform, Load

Quelle: <https://www.astera.com/de/Typ/Blog/etl-prozess-und-schritte/> , Februar 2020

- Bei der Extraktion wird in der Regel ein Ausschnitt der Daten aus den Quellen extrahiert und für die Transformation bereitgestellt.
- Die Quellen können aus verschiedenen Informationssystemen mit verschiedenen Datenformaten und -strukturen bestehen.
- Bei der Extraktion findet eine Schematransformation vom Schema der Quelldaten in das Schema des Arbeitsbereichs statt.

- Um das Data-Warehouse mit aktuellen Daten zu versorgen, muss die Extraktion regelmäßig stattfinden.
- Dies kann synchron mit den Quellen oder asynchron geschehen.
 - ▶ Bei synchroner Extraktion wird jede Änderung am Quellsystem sofort an das Data-Warehouse propagiert. Dieser Ansatz ermöglicht das Konzept des Real-Time-Data-Warehousing, welches den Bedarf nach sofort verfügbaren Daten unter Wahrung der Trennung von operativen und auswertenden Systemen deckt.
 - ▶ Die asynchrone Extraktion kann periodisch, ereignisgesteuert oder anfragegesteuert erfolgen.

- periodisch: periodisch Die Quelle erzeugt in regelmäßigen Abständen Auszüge ihrer Daten, die regelmäßig abgefragt werden.
- ereignisgesteuert: Die Quelle erzeugt bei bestimmten Ereignissen – beispielsweise nach einer bestimmten Anzahl von Änderungen – einen Auszug.
- anfragegesteuert: Die Quelle stellt Auszüge erst auf Anfrage bereit.

- Die Transformation besteht im Wesentlichen aus der Anpassung der Daten an die vorgegebenen Zielstrukturen (Schema-Mapping).
- Unter Transformation fällt hierbei auch die aufwändige Datenbereinigung, falls diese durchgeführt werden soll. Es gibt Firmen, welche den Ist-Zustand der Anwendungen im DWH belassen wollen. Andere Firmen wiederum wollen einen bereinigten SPOT. Die Transformation findet in einem eigenen Arbeitsbereich (Staging-Area) statt.

- Syntaktische Transformation:
 - ▶ Hier geht es um die Verbesserung, Umsetzung oder Korrektur der Daten basierend auf formalen Aspekten.
 - ▶ Die Daten werden gemäß der im Zielsystem notwendigen und angewandten Syntax modifiziert.
 - ▶ Ein Beispiel dafür ist die Anpassung von Datentypen (z. B. numerische Darstellung des Tagesdatums YYYYMMDD hin zu dem standardisierten Datumsformat vom Datentyp date)

Hierbei werden die Daten auf inhaltliche Aspekte überprüft und wenn nötig modifiziert und angereichert. Hierunter fallen

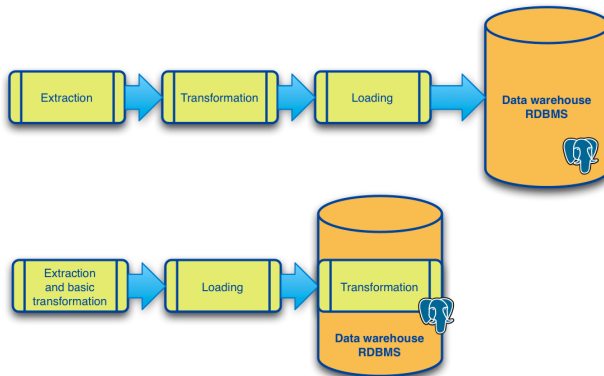
- Eliminierung von Duplikaten (Objektidentifizierung)
- Schlüssel Anpassung (z.B. unterschiedliche Ländercodierungen hin zu DIN ISO Ländercodes)
- Anpassung von Datenwerten (z. B. unterschiedliche Codierung des Geschlechts wie 1 (weiblich), 2 (männlich) hin zu f (female) und m (male))
- Umrechnung von Maßeinheiten (z.B. unterschiedliche Volumina Gallone zu Liter etc.)
- Aggregation (z. B. Einzelumsätze eines Vertriebsprodukts hin zu monatlichen Umsätzen je Vertriebsprodukt)
- Anreicherung der gelesenen Daten aus den Quellsystemen mit Zusatzinformation.

- Beim Laden müssen die Daten aus dem Arbeitsbereich in das Data-Warehouse eingebracht werden.
- Das Laden soll in der Regel möglichst effizient geschehen, so dass die Datenbank während des Ladens nicht oder nur kurz blockiert wird und ihre Integrität gewahrt wird.
- Im Hinblick auf die Integration der Daten im Data-Warehouse ist eine weitere Schematransformation vom Schema des Arbeitsbereichs in das Schema des Data-Warehouses notwendig.

Auch wenn man ETL-Prozesse mit eigenen Programmen umsetzen kann, sprechen folgende Gründe für den Einsatz von Standardwerkzeugen:

- Jedes Standardwerkzeug unterstützt den Zugriff auf die gängigen Datenbanksysteme sowie ERP- und Dateisysteme.
- Meist sind auch für High-Performance-Loading die entsprechenden Voraussetzungen bereits im Standardwerkzeug implementiert. Eine genaue Kenntnis der Mechanismen der Zielsysteme entfällt dadurch meistens.
- Entwicklung und Wartung der ETL-Prozesse sind in der Regel durch visualisierende Standardwerkzeuge einfacher und kostengünstiger durchzuführen als bei Systemen auf Basis entwickelter Programme unter Verwendung von Programmiersprachen.

ETL or ELT?



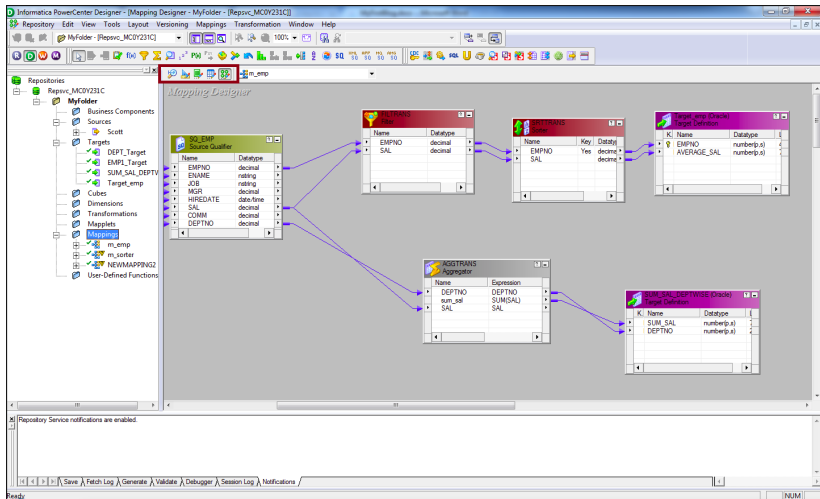
Kommerziell:

- SAS Institute
- IBM Information Server
- Informatica Power Center
- SAP-Business Objekt
- Altova Map Force
- Oracle Warehouse Builder
- Microsoft SQL Server Integration Services

Open-Source:

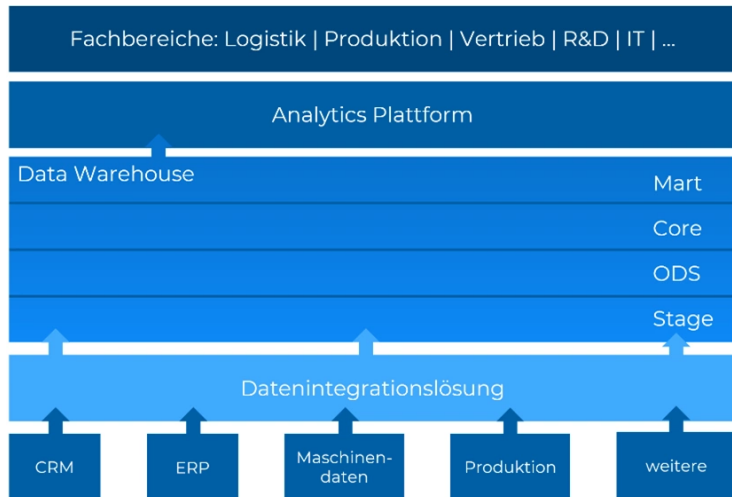
- Talend Open Studio
- Kettle Pentaho Data Integration
- Scriptella ETO

Anwendung Informatica Power Center



Quelle: <https://senturus.com/wp-content/uploads/2018/06/Data-modeling-in-Informatica-PowerCenter.png> , März 2020

Datawarehouse und Logistik



TransLog GmbH

- Mittelständisches Transportlogistikunternehmen
- Bietet nationale und internationale Transportdienstleistungen an
- Über 700 Mitarbeiter
- Flotte von 200 LKWs und 20 Lagerhäusern

- Ein Data Warehouse (DWH) ist ein zentraler Speicher für Daten.
- Unterstützt die Entscheidungsfindung durch konsolidierte, saubere und konsistente Daten.

- **Datenintegration:** Daten aus verschiedenen Quellen (ERP, TMS, WMS, GPS, Lieferanten).
- **Datenqualität:** Unterschiedliche Formate, Inkonsistenzen, Duplikate.
- **Echtzeitanalysen:** Bedarf an Echtzeitdaten.
- **Berichterstattung:** Zeitaufwändige Erstellung von Berichten.

- **Datenkonsolidierung:** Zentraler Speicher für alle Datenquellen.
- **Datenqualität:** Konsistenz, Genauigkeit und Aktualität der Daten.
- **Echtzeitanalysen:** Bereitstellung von Daten in Echtzeit.
- **Automatisierte Berichte:** Erstellung und Verteilung von Berichten.

- **Datenquellen:** ERP, TMS, WMS, GPS-Tracking, Lieferantendaten.
- **ETL-Prozess:**
 - ▶ **Extract:** Datenextraktion.
 - ▶ **Transform:** Datenbereinigung und -konsolidierung.
 - ▶ **Load:** Laden der Daten in das DWH.
- **Datenbank:** Relationale Datenbank oder Cloud-DWH (z.B. Amazon Redshift).
- **Analyse und Reporting:** BI-Tools (z.B. Tableau, Power BI).

- **Bedarfsanalyse und Planung:** Identifikation der Datenquellen und Anforderungen.
- **Datenintegration:** Entwicklung von ETL-Prozessen, Datenqualitätsprüfungen.
- **Datenmodellierung:** Entwurf eines Datenmodells (Dimensionen, Fakten).
- **Implementierung des DWH:** Aufbau der Infrastruktur, Implementierung der ETL-Prozesse.
- **Datenanalyse und Reporting:** Einrichtung von BI-Tools, Erstellung von Dashboards.

- **Verbesserte Entscheidungsfindung:** Konsolidierte und genaue Daten.
- **Effizienzsteigerung:** Optimierte Routenplanung, Lagerverwaltung.
- **Kostenreduktion:** Reduktion von manuellen Prozessen und Fehlern.
- **Transparenz:** Bessere Nachverfolgbarkeit und Berichterstattung.

Table of contents

1. Data Integration
2. Data Warehousing
3. Data Integration / Extract Transform Load (ETL)
4. Datalake

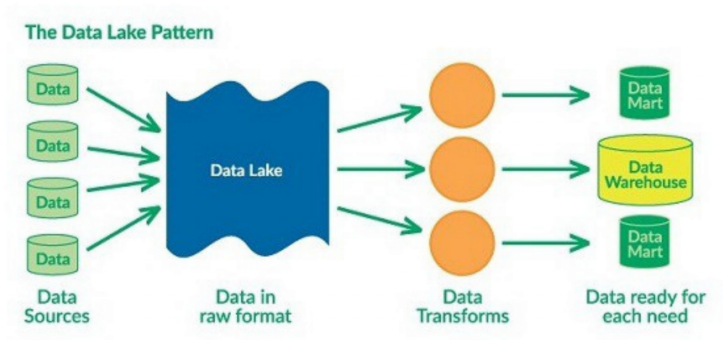
Definition Datalake

- Datenspeicher, der die Daten im Rohformat hält.
- Die Daten werden erst in ein anderes Format umgewandelt, wenn sie verwendet werden (Schema on Read).

Historie:

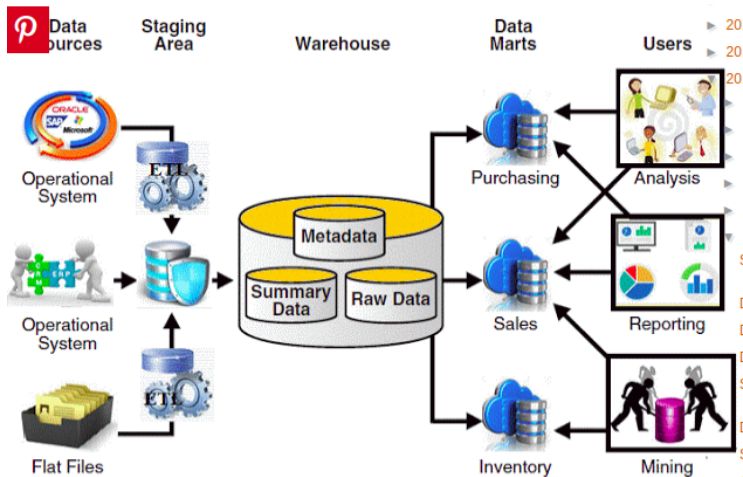
- Im Vergleich zum Data Warehouse ist die Datenspeicherung im Datalake kostengünstiger, da die Daten weniger aufbereitet werden müssen.
- Durch die Heterogenität der Daten können Zusammenhänge zwischen den Daten schnell erkannt und genutzt werden.

- Ein Data Lake speichert die Daten in einer flachen und unbearbeiteten Struktur (häufig über komprimierte Dateien auf verteilte file-Systeme oder als Datenobjekte in Object Storage Systemen wie minio¹). *Dadurch kann nahezu eine beliebige Menge von Daten in dem Data Lake gespeichert werden.*
- Data Lakes können Wettbewerbsvorteile schaffen: Durch die genaue Analyse aller Verkaufstransaktionen, verbunden mit Logdateien, Kundenmeinungen usw., kann die Preis- und Angebotspolitik entscheidend verbessert werden.
- Data lakes sind aufgrund des Schema on Demands schnell, daher eignen sie sich für für Echtzeitanwendungen.



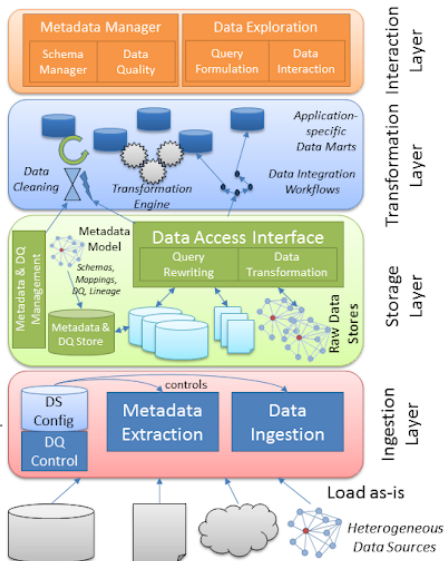
Quelle: <http://www.datavirtualizationblog.com/role-data-virtualisation-data-lake/>, Februar 2020

Architektur Datalake



Quelle: <http://www.sql-datatools.com/2016/07/modern-data-warehouse-in-sql-server-2016.html>, Februar 2020

Architektur Datalake



Konzept

- Zentraler Speicher für große Datenmengen
- Speicherung strukturierter und unstrukturierter Daten
- Basis: Hadoop Distributed File System (HDFS)

Architektur

- NameNode verwaltet Metadaten
- DataNodes speichern Datenblöcke
- Verteilte Speicherung im Cluster
- Verarbeitung mit Apache Spark oder Hive

Technische Eigenschaften

- Verteiltes Dateisystem
- Blockbasierte Speicherung
- Replikation zur Ausfallsicherheit
- Hoher Durchsatz für Batch Processing

Vorteile

- Sehr gute Big-Data-Performance
- Bewährtes Hadoop-Ökosystem
- Datenlokalität
- Gute Skalierbarkeit

Nachteile

- Hoher Administrationsaufwand
- Komplexe Infrastruktur
- Weniger cloud-native
- Echtzeitverarbeitung schwieriger

Konzept

- Moderner cloud-nativer Data Lake
- Object Storage mit S3-kompatibler API
- Speicherung als Objekte in Buckets

Architektur

- Zugriff über HTTP/S3
- Trennung von Storage und Compute
- Integration mit Spark, Trino und Iceberg

Technische Eigenschaften

- Object Storage statt Dateisystem
- Horizontale Skalierung
- Erasure Coding
- Kubernetes- und Cloud-optimiert

Vorteile

- Einfache Skalierung
- Geringerer Betriebsaufwand
- Cloud-native Architektur
- Sehr gut für KI und Analytics

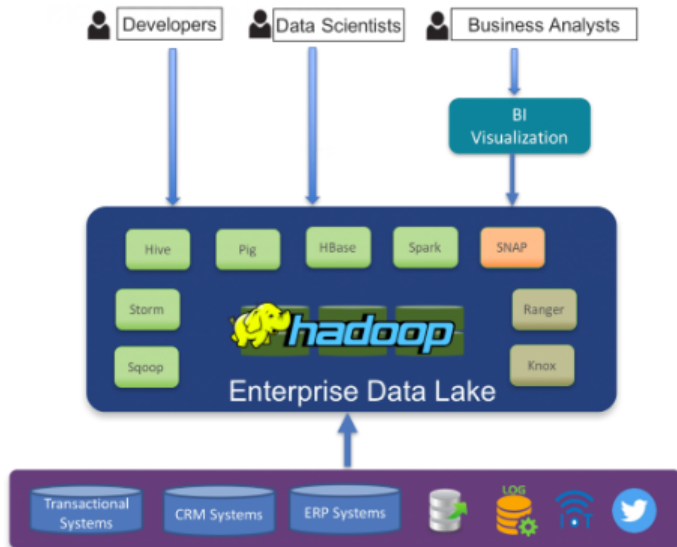
Nachteile

- Kein klassisches Dateisystem
- Netzwerkperformance wichtig
- Teilweise Anpassungen bei Legacy-Tools

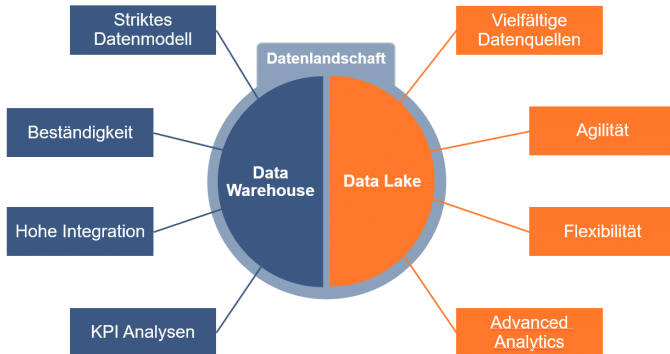
Vergleich HDFS vs. MinIO

Merkmal	HDFS	MinIO
Speicherprinzip	Dateisystem	Object Storage
Architektur	Hadoop-basiert	Cloud-native
Zugriff	HDFS API	S3 API
Skalierung	Clusterbasiert	Sehr flexibel
Einsatzgebiet	Batch Analytics	Cloud / KI / Echtzeit
Betriebsaufwand	Hoch	Niedriger

Technische Umsetzung eines Datalakes mit Hadoop



Datalake versus Data Warehouse



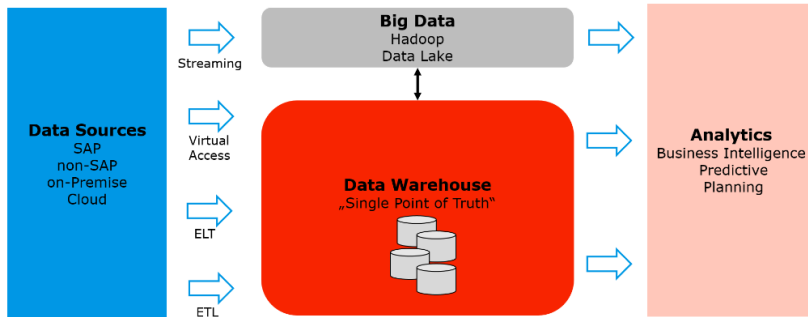
Quelle: <https://www.alexanderthamm.com/de/artikel/grundlagen-anwendungsfaelle-data-lake/>, Februar 2020

Datalake versus Data Warehouse

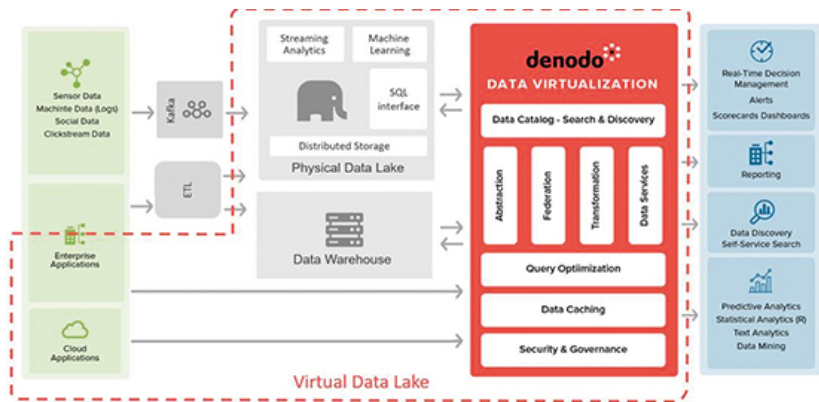
DATA WAREHOUSE	vs.	DATA LAKE
structured, processed	DATA	structured / semi-structured / unstructured, raw
schema-on-write	PROCESSING	schema-on-read
expensive for large data volumes	STORAGE	designed for low-cost storage
less agile, fixed configuration	AGILITY	highly agile, configure and reconfigure as needed
mature	SECURITY	maturing
business professionals	USERS	data scientists et. al.

Quelle: <https://www.pinterest.de/pin/522628731741388885/?lp=true>, Februar 2020

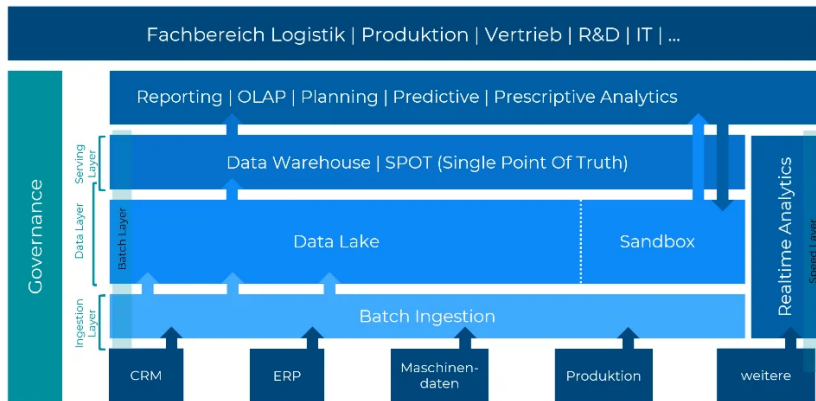
Datalake und Data Warehouse



Datalake und Data Warehouse



Datalake in der Logistik



- Datensilos in verschiedenen Abteilungen
- Schwierigkeit bei der Datenintegration
- Begrenzte Datenanalysefähigkeiten
- Hohe Kosten für Datenlagerung und -verwaltung

Was ist ein Data Lake?

- Zentrale Speicherung aller Unternehmensdaten
- Unterstützung von strukturierten (ohne explizites Schema, anders als im Datawarehouse. Tabellendaten kommen z.B. als komprimierte .csv Tabellendaten vor) und unstrukturierten Daten
- Flexibilität für Datenanalyse und Machine Learning
- Skalierbare und kosteneffiziente Lösung

- Datenquellen: ERP-Systeme, CRM, IoT-Geräte, etc.
- Datenspeicherung: Cloud-basierte Lösungen (z.B. AWS S3, Azure Data Lake)
- Datenverarbeitung: ETL-Prozesse, Echtzeit-Streaming
- Datenanalyse: BI-Tools, Data Science Plattformen

Data Lake vs. Data Warehouse in digitalen Supply Chains

Kriterium	Data Lake	Data Warehouse
Datenarten	Strukturierte, semi-strukturierte und unstrukturierte Daten	Vor allem strukturierte Daten
Flexibilität	Sehr hohe Flexibilität durch Speicherung roher Daten	Festes Schema und definierte Datenmodelle
Analytics	KI, Machine Learning und Big-Data-Analytics	Business Intelligence und Reporting
Echtzeitfähigkeit	Sehr gut für Streaming- und IoT-Daten	Meist Batch-orientiert
Datenqualität	Teilweise heterogene Datenqualität	Hohe Konsistenz und Validierung
Praxisbeispiel	Echtzeit-Sensordaten aus Transportfahrzeugen, GPS-Tracking und Logdaten werden zentral gespeichert und mit KI analysiert	Tägliche Analyse von Lieferzeiten, Lagerbeständen und Umsatzkennzahlen für Management-Reports
Vorteile	Hohe Skalierbarkeit und flexible Datennutzung	Hohe Datenqualität und schnelle Standardreports
Nachteile	Komplexere Verwaltung und Governance	Weniger flexibel bei neuen Datentypen